

Computer Vision and Multimodal Machine Learning - comp6001-assignment-1

Ibrahim Iftekhar Khan - a1986169

CONTENTS

1. Introduction

- Motion blur problem
- Why image restoration matters
- Why object detection matters
- Goal: test whether deblurring improves detection

2. Dataset

- GoPro dataset
- Paired blurred and sharp images
- Subset of 100 images used

3. Image Deblurring Method

- Richardson–Lucy
- Kernel size
- Iterations
- Why chosen

4. Deblurring Evaluation

- Visual comparison
- PSNR
- SSIM
- Runtime
- Discussion

5. Object Detection on Blurred vs Deblurred vs Sharp

- YOLOv8
- Pretrained model
- Detection counts
- Confidence
- Detection per image plot
- Failure cases

6. Dataset Preparation and Training

- Train/val/test split
- Why are deblurred images used
- Fine-tuning YOLO

7. Final Evaluation

- mAP
- Precision
- Recall
- Comparison table

8. Discussion

- Did deblurring help?
- Trade-offs
- Limitations
- Future work

9. Conclusion

- Final summary

INTRODUCTION

Motion blur is a common problem in images captured using handheld cameras, moving objects, or low-light conditions. Motion blur reduces image sharpness and removes important visual details such as edges and textures. This degradation in image quality can negatively affect tasks such as object detection, where models rely on clear visual features to correctly identify objects.

Image restoration techniques, like image deblurring, are used to get images from blurry ones. These techniques try to fix the blur and get back the details that were lost. It is not always easy to say if making the image look better also helps with other tasks, like finding objects in the image. The main thing we want to do in this project is to see how image restoration affects how well we can find objects in an image. We are going to take images and use an old method to make them sharp again. Then we will try to find objects in the images, the sharp images we made, and the original sharp images.

2. AI Usage and Git Version Control

2.1 AI Usage

Artificial Intelligence tools helped with coding, debugging, and documentation in this project. They were used to calculate evaluation metrics like PSNR and SSIM. I reviewed all AI interactions. Made changes where needed to ensure they were correct and made sense.

All AI prompts and outputs are documented in the `ai_logs/` file. This was required by the assignment guidelines. The AI was used to boost productivity and help with debugging. However, I made all decisions and interpretations myself, not the AI.

2.2 Git Version Control

Git version control was used throughout the project to track progress and maintain version history. The repository was organized into folders for data, notebooks, outputs, reports, and AI logs. Atomic and descriptive commits were made at each major stage of the project, including project setup, dataset preparation, deblurring implementation, detection experiments, pseudo-label generation, model training, and final evaluation.

The commit history shows the development process of the project and ensures reproducibility. Git was also used to manage different versions of notebooks, outputs, and documentation files throughout the project.

3. Methodology

The overall workflow of the project is: blurred image → deblurring → object detection → pseudo-label generation → model fine-tuning → evaluation and comparison.

3.1 Dataset

The GoPro Deblur Dataset was used for this project. The dataset has paired images: blurred images and their corresponding sharp (ground truth) images. The images were captured under real motion blur conditions. Each blurred image has a matching sharp image, which allows both visual evaluation and quantitative evaluation of image restoration methods.

Due to computational limitations, only a subset of 100 matched blurred–sharp image pairs was used for the experiments.

The dataset was organized into three folders:

- Blurred images
- Deblurred images
- Sharp images

3.2 Image Deblurring Method

Image deblurring was performed using the Richardson–Lucy deconvolution algorithm, which is a classical iterative method for image restoration. The algorithm estimates the sharp image by iteratively reducing the blur effect using a motion blur kernel. In this project, a motion blur kernel of size 15 was used, and the algorithm was run for 10 iterations for each image. The output of the algorithm was a restored grayscale image. The performance of the deblurring method was evaluated using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) by comparing the restored images with the corresponding sharp ground truth images.

The Richardson–Lucy algorithm was chosen because it is suitable for motion blur and works well when the blur kernel is approximately known. Unlike deep learning-based

restoration methods, the Richardson–Lucy method does not require training data and can be applied directly to blurred images. This makes it suitable for this project, where a classical restoration method is required for comparison with deep learning-based object detection.

3.3 Object Detection

Object detection was performed using the YOLOv8 model, which is a deep learning-based object detection algorithm. A pre-trained YOLOv8n model was used to perform object detection on blurred images, deblurred images, and sharp images. The number of detections and average detection confidence were recorded and compared across the three image types to evaluate the effect of image restoration on object detection performance.

3.4 Pseudo-Label Generation

The GoPro dataset does not contain object detection annotations. Therefore, pseudo-labels were generated using the pre-trained YOLOv8 model on the sharp images. These pseudo-labels were used as approximate ground truth annotations for training the object detection model. This approach allows the model to be fine-tuned without manually labeling the dataset.

3.5 Model Training

The YOLOv8 model was fine-tuned using the restored (deblurred) images and the pseudo-labels generated from the sharp images. The dataset was split into 80% training and 20% validation sets. The model was initially trained for 10 epochs using a YOLOv8n model to establish a baseline. A stronger YOLOv8s model was then trained for 30 epochs with data augmentation to improve performance. Training performance was evaluated using standard object detection metrics, including precision, recall, and mean Average Precision (mAP).

One limitation of this project is that detection performance was measured using detection counts and mAP rather than manually verified ground truth bounding boxes, since pseudo-labels were used.

4. Deblurring Results

4.1 Visual Evaluation

The Richardson–Lucy deconvolution algorithm was applied to restore blurred images from the GoPro dataset. Visual comparison was performed between blurred images, restored images, and the corresponding sharp ground truth images. The restored images showed improved edge sharpness and better visibility of object boundaries compared to the blurred images. However, the restored images did not fully match the sharp ground truth images, and some noise and artifacts were observed in the restored images. This is expected because image deblurring is an ill-posed problem, and some information lost during motion blur cannot be perfectly recovered.



Figure 4.1: Comparison between blurred image, restored image, and sharp ground truth image.

4.2 Quantitative Evaluation

Quantitative evaluation of the deblurring performance was performed using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM). These metrics compare the restored image with the corresponding sharp ground truth image. PSNR measures the pixel-level reconstruction quality, while SSIM measures structural similarity between images.

Deblurring Performance Metrics

Metric	Value
Average PSNR	24.83 dB
Average SSIM	0.7467
Average Processing Time	32.36 seconds per image

The PSNR value of 24.83 dB indicates moderate reconstruction quality, while the SSIM value of 0.7467 indicates that the restored images preserve a significant amount of structural information from the sharp images. However, the processing time of approximately 32 seconds per image shows that the Richardson–Lucy algorithm is computationally expensive and may not be suitable for real-time applications.

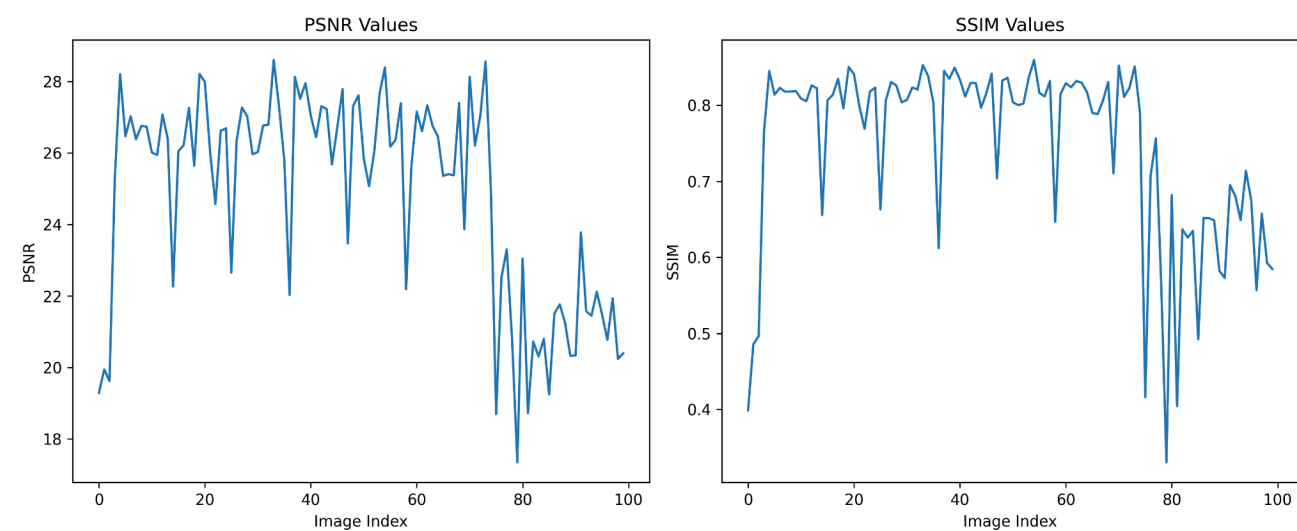


Figure 4.2: PSNR and SSIM values across the dataset.

5. Object Detection Comparison

5.1 Detection Using Pre-trained YOLOv8

Object detection was first performed using a pre-trained YOLOv8n model on blurred images, restored images, and sharp images. The total number of detections and average confidence scores were recorded for each image type to compare detection performance.

Detection Results Using Pre-trained Model

Image Type	Total Detections	Average Confidence
Blurred Images	1011	0.6098
Deblurred Images	806	0.5786
Sharp Images	1042	0.6151

The results show that the object detector produced the highest number of detections on the sharp images, followed by blurred images, and the lowest number of detections on the restored images. The average confidence was also highest for sharp images and lowest for restored images. This indicates that although the restoration algorithm improved visual image quality, it did not improve object detection performance when using a pre-trained detector. This may be due to artifacts and noise introduced during the restoration process.



Figure: Number of detections per image for blurred, restored, and sharp images.

5.2 Failure Case Analysis

A failure case analysis was performed by comparing the number of detections per image across blurred, restored, and sharp images. In some cases, objects detected in blurred images were not detected in restored images, which suggests that restoration artifacts may have negatively affected the object detector. However, sharp images consistently produced the best detection results, indicating that image quality plays an important role in object detection performance.

Table 5.2.1 shows failure cases where the number of detections on restored images is significantly lower than on blurred images. The negative values in the “Deblurred – Blurred” column indicate that fewer objects were detected after image restoration. However, the sharp images still produced higher detection counts than the restored images. This suggests that the restoration process introduced artifacts or altered image features in a way that negatively affected the object detector. These failure cases demonstrate that image restoration does not always improve object detection performance and may sometimes reduce detection accuracy when using a pre-trained detector.

Image Index	Blurred Detections	Deblurred Detections	Sharp Detections	Deblurred – Blurred	Sharp – Deblurred
26	19	11	20	-8	9
47	15	7	13	-8	6
69	14	6	14	-8	8
58	14	7	13	-7	6
27	19	12	14	-7	2
59	13	7	14	-6	7
31	17	11	13	-6	2
54	13	7	13	-6	6
63	21	15	16	-6	1
18	16	10	15	-6	5

Table 5.2.1: Failure Case Detection Comparison

The failure cases show that the deblurring algorithm does not always improve object detection performance. In several images, the number of detections decreased after restoration compared to blurred images. This may be due to artifacts introduced during the deblurring process, which can distort object features and reduce detector performance. However, the sharp images consistently produced higher detection counts, which confirms that image quality is still an important factor for object detection.

6. Model Training and Fine-Tuning

6.1 Pseudo-Label Generation

The GoPro Deblur dataset did not contain object detection annotations. Therefore, pseudo-labels were generated using a pre-trained YOLOv8 model on the sharp images. The bounding boxes on the sharp images were saved and used as approximate ground truth annotations. This approach allowed the model to be trained without manual labeling of the dataset.

6.2 Dataset Preparation

The dataset was prepared using the restored (deblurred) images and the pseudo-labels generated from the sharp images. The dataset was split into training and validation sets using an 80:20 split. The training set contained 80 images and the validation set contained 20 images. The dataset was organized in the YOLO format with separate folders for images and labels for both training and validation sets.

6.3 Model Training

The YOLOv8 model was fine-tuned on the restored image dataset using pseudo-labels generated from the sharp images. In the initial experiment, a YOLOv8n model was trained for 10 epochs to establish a baseline fine-tuned detector. To improve detection performance, a stronger YOLOv8s model was then trained for 30 epochs using data augmentation techniques such as scaling, translation, brightness adjustment, horizontal flipping, mosaic augmentation, and mixup. The dataset was split into 80% training and 20% validation sets. Training was performed using Google Colab GPU. The improved training setup resulted in higher recall and mean Average Precision (mAP), indicating better overall detection performance.

7. Final Evaluation

7.1 Detection Performance After Fine-Tuning

After fine-tuning the YOLOv8 model on the restored image dataset using pseudo-labels, the model was evaluated on the validation set. The performance of the fine-tuned model was measured using standard object detection metrics, including precision, recall, and mean Average Precision (mAP).

Fine-Tuned Model Yolov8n Performance(10 epochs)

Metric	Value
Precision	0.8545
Recall	0.4376
mAP@50	0.5015
mAP@50-95	0.3938

Fine-Tuned Model Yolov8s Performance (30 epochs)

Metric	Value
Precision	0.6817
Recall	0.6383

mAP@50	0.7070
mAP@50-95	0.5631

The improved YOLOv8s model achieved a precision of 0.6817, recall of 0.6383, mAP@50 of 0.7070, and mAP@50:95 of 0.5631. Compared to the initial fine-tuned model, precision decreased slightly, but recall and mAP improved significantly. This indicates that the improved model was able to detect more objects overall, although it also produced more false positives. Since mAP increased significantly, the overall detection performance improved after training with augmentation and a larger model.

7.2 Comparison Before and After Fine-Tuning

Detection Counts Comparison

Model	Blurred	Deblurred	Sharp
Pre-trained YOLO	1011	806	1042
Fine-tuned YOLO	1111	901	1149
Improved Fine-tuned model YOLOv8s	1139	1163	1189

The improved fine-tuned YOLOv8s model produced the highest number of detections across all image types. Detections on blurred images increased to 1139, detections on restored images increased to 1163, and detections on sharp images increased to 1189. Most importantly, the improved model produced more detections on restored images than on blurred images. This indicates that training a stronger detector with data augmentation

helped the model adapt to restored image characteristics and improved detection performance on restored images.

8. Discussion

The results show that image restoration does not always improve object detection performance when using a pre-trained model. Although the Richardson–Lucy deblurring method improved image quality and produced reasonable PSNR and SSIM values, the YOLOv8 model detected fewer objects on restored images compared to blurred images. This may be due to artifacts and noise introduced during the deblurring process, which can affect deep learning models.

However, after fine-tuning YOLOv8 on restored images using pseudo-labels, detection performance improved on blurred, restored, and sharp images. The improvement was most significant for restored images, showing that the model learned the features of restored images during training.

These results suggest that image restoration alone is not enough; object detection models need to be trained or fine-tuned on restored images to benefit from restoration. A limitation of this project is the small dataset and the use of pseudo-labels instead of manual annotations.

In future work, larger datasets, manual annotations, and advanced deep learning-based deblurring methods could be used. The fine-tuned model also showed a trade-off between precision and recall, where recall and mAP increased but precision decreased slightly, indicating more detections but also more false positives. Overall, the results support that restoration improves detection only when the detector is trained on restored images.

9. Conclusion

This project investigated the effect of image restoration on object detection performance using blurred, restored, and sharp images from the GoPro dataset. A classical image deblurring method, Richardson–Lucy deconvolution, was used to restore blurred images, and the restoration quality was evaluated using PSNR and SSIM metrics. Initial experiments with a pre-trained YOLOv8 detector showed that deblurring alone did not improve object detection

performance. However, after fine-tuning a stronger YOLOv8s detector on restored images using pseudo-labels and data augmentation, detection performance improved substantially. The improved model achieved higher recall and mAP and produced more detections on restored images than on blurred images. These results show that image restoration can improve object detection performance when the detector is trained on restored images. Therefore, restoration alone is not sufficient, but restoration combined with model fine-tuning can improve detection performance.